

VPN

A **virtual private network (VPN)** is an [overlay network](#) that uses [network virtualization](#) to extend a [private network](#) across a public network, such as the [Internet](#), via the use of [encryption](#) and [tunneling protocols](#). In a VPN, a tunneling protocol is used to transfer network messages from one [network host](#) to another.

WireGuard is a [communication protocol](#) and [free and open-source](#) software that implements encrypted [virtual private networks](#) (VPNs). It aims to be lighter and better performing than [IPsec](#) and [OpenVPN](#), two common [tunneling protocols](#). The WireGuard protocol passes traffic over [UDP](#).

sudo apt update

sudo apt install wireguard -y

wireguard-tools → wg, wg-quick

wg genkey

- Generates a **256-bit private key**
- Uses Curve25519 cryptography - In [cryptography](#), **Curve25519** is an [elliptic curve](#) used in [elliptic-curve cryptography](#) (ECC) offering 128 [bits of security](#) (256-bit [key size](#))

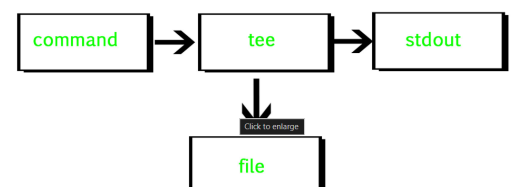
Elliptic-curve cryptography (ECC) is an approach to [public-key cryptography](#) based on the [algebraic structure](#) of [elliptic curves](#) over [finite fields](#). ECC allows smaller keys to provide equivalent security, compared to cryptosystems based on modular exponentiation in [finite fields](#), such as the [RSA cryptosystem](#) and [ElGamal cryptosystem](#).^[1]

The tee command reads standard input and writes it to both standard output and one or more files at the same time.

Writes input → file

ALSO prints it to terminal

tee server_private.key



wg pubkey

- Takes private key as input
- Outputs corresponding public key > server_public.key

genkey Generates a random [private](#) key in base64 and prints it to standard output.

genpsk Generates a random [preshared](#) key in base64 and prints it to standard output.

pubkey Calculates a [public](#) key and prints it in base64 to standard output from a corresponding [private](#) key (generated with [genkey](#)) given in base64 on standard input.

A private key and a corresponding public key may be generated at once by calling:

```
$ umask 077
```

```
$ wg genkey | tee private.key | wg pubkey > public.key
```

sudo nano /etc/wireguard/wg0.conf

CONFIGURATION FILE FORMAT EXAMPLE

[top](#)

This example may be used as a model for writing configuration files, following an INI-like syntax. Characters after and including a '#' are considered comments and are thus ignored.

```
[Interface]
PrivateKey = yAnz5TF+lXXJte14tji3z1MNq+hd2rYUIgJBgB3fBmk=
ListenPort = 51820

[Peer]
PublicKey = xTIBA5rboUvnH4htodjb6e697QjLERT1NAB4mZqp8Dg=
Endpoint = 192.95.5.67:1234
AllowedIPs = 10.192.122.3/32, 10.192.124.1/24

[Peer]
PublicKey = TrMvSoP4jYQ1Y6RIzBgbssQqY3vxI2Pi+y7110WWXX0=
Endpoint = [2607:5300:60:6b0::c05f:543]:2468
AllowedIPs = 10.192.122.4/32, 192.168.0.0/16

[Peer]
PublicKey = gN65BkIKy1eCE9pP1wdc8ROUtkHLF2PfAqYdyYBz6EA=
Endpoint = test.wireguard.com:18981
AllowedIPs = 10.10.10.230/32
```

```
GNU nano 6.2 /etc/wireguard/wg0.conf *
[Interface]
PrivateKey = QF41FHPAezrm1thS9/mM+9y0dl7ofEcP7U9/CzZD6l8=
Address = 10.0.0.1/24
ListenPort = 51820

PostUp = iptables -t nat -A POSTROUTING -o <interface> -j MASQUERADE
PostDown = iptables -t nat -D POSTROUTING -o <interface> -j MASQUERADE
|
[Peer]
PublicKey=KmU3oHCBEQivlPqX4ygVlwbwSom0Q9HH6fulExr902E=
AllowedIPs = 10.0.0.2/32
```

NAT (Network Address Translation) = Convert private IP addresses into public IP addresses (and vice versa)

Before:

SRC = 10.0.0.2

DST = 8.8.8.8

After NAT:

SRC = 20.x.x.x (your server public IP)

DST = 8.8.8.8

SNAT (Source NAT)

- Changes **source IP**
- Used for outgoing traffic

DNAT (Destination NAT)

- Changes **destination IP**
- Used in port forwarding

`iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE`

[Use NAT table (special table for IP translation) -t nat]

Packet flow : Incoming → PREROUTING → Routing → POSTROUTING → Outgoing

POSTROUTING = just before packet leaves server

-o eth0 : Apply rule only when leaving via **eth0**

-j MASQUERADE {Replace source IP with server's IP automatically}

```
$ sudo iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth1 -j MASQUERADE
```

The POSTROUTING rule is further customized in this step by specifying a particular interface (eth1) and a source IP address range (192.168.1.0/24). This rule ensures that only traffic from the specified source IP address range going out through the specified interface undergoes masquerading.

The **SNAT** target requires you to give it an IP address to apply to all the outgoing packets. The **MASQUERADE** target lets you give it an interface, and whatever address is on that interface is the address that is applied to all the outgoing packets. In addition, with **SNAT**, the kernel's connection tracking keeps track of all the connections when the interface is taken down and brought back up; the same is not true for the **MASQUERADE** target.

Netfilter is the kernel-level packet processing framework in Linux

User space tools (iptables, nftables)



Netfilter (inside kernel)



Actual packet processing

(ip tables are just tools, netfilter is the real engine)

Netfilter has different **tables**, each for a purpose: (NAT table, filter table, raw table, mangle table)

WireGuard → creates tunnel

Netfilter → routes & NATs traffic

Server	[Interface]	Server PrivateKey
Server	[Peer]	Client PublicKey
Client	[Interface]	Client PrivateKey
Client	[Peer]	Server PublicKey

My interfaces other than eth0 :

Interface	Why not
lo	loopback (internal only)
docker0	Docker network
veth...	container virtual links
enP10482s1	slave of eth0 (don't use directly)

Eth0 has IP **10.0.0.4** → VM's network interface

```
hekhek@hekhek:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

sudo wg-quick up wg0

Wg-quick : High-level helper tool for WireGuard

ip link add

ip address add

wg set

iptables rules

==wg-quick

Wg0 : Interface name that maps to wg0.conf

[Windows Laptop]

↓ (VPN tunnel)

[My VM (WireGuard Server)]

↓

[Internet]

Windows sends encrypted traffic → VM

VM decrypts

NAT converts IP

Sends to internet

Response comes back same way

wg genkey | tee client2_private.key | wg pubkey > client2_public.key ; generated two more keys on VM

WireGuard keys are just text (crypto keys) - we can create them from anywhere

```
hekhek@hehek:~$ sudo ufw allow 51820
Rule added
Rule added (v6)
hekhek@hehek:~$ sudo ufw allow 51820/udp
Rule added
Rule added (v6)
```

```
hekhek@hehek:~$ sudo wg-quick down wg0
sudo wg-quick up wg0
[#] ip link delete dev wg0
[#] iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
hekhek@hehek:~$ sudo wg
interface: wg0
  public key: KmU3oHCBEQivlPqX4ygVlwbwSom0Q9HH6fulExr902E=
  private key: (hidden)
  listening port: 51820

peer: Sv0QW42WdNrPmVHaJhrE3AP7rFhk1M9vD6vtND0ydSE=
  endpoint: 1.39.47.23:34116
  allowed ips: 10.0.0.2/32
  latest handshake: 7 seconds ago
  transfer: 180 B received, 92 B sent
hekhek@hehek:~$
```

```
PS C:\Users\Lenovo> ping 10.0.0.1

Pinging 10.0.0.1 with 32 bytes of data:
Reply from 10.0.0.1: bytes=32 time=398ms TTL=64
Reply from 10.0.0.1: bytes=32 time=185ms TTL=64
Reply from 10.0.0.1: bytes=32 time=218ms TTL=64
Reply from 10.0.0.1: bytes=32 time=205ms TTL=64

Ping statistics for 10.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 185ms, Maximum = 398ms, Average = 251ms
PS C:\Users\Lenovo>
```

WireGuard VPN Configuration – Complete Process Documentation

1. Objective

To configure a secure VPN server using WireGuard on a cloud VM and create at least two client configurations, enabling secure communication between client devices and the server.

2. Initial Setup

- Created an Ubuntu VM on Azure.
- Installed WireGuard:

```
sudo apt update
```

```
sudo apt install wireguard -y
```

3. Key Generation

Generated key pairs for:

- Server
- Client 1
- Client 2

Commands used:

```
wg genkey | tee server_private.key | wg pubkey > server_public.key
```

```
wg genkey | tee client1_private.key | wg pubkey > client1_public.key
```

```
wg genkey | tee client2_private.key | wg pubkey > client2_public.key
```

4. Server Configuration

Created config file:

```
sudo nano /etc/wireguard/wg0.conf
```

Configured:

[Interface]

PrivateKey = <server_private.key>

Address = 10.0.0.1/24

ListenPort = 51820

PostUp = iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

PostDown = iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE

[Peer]

PublicKey = <client1_public.key>

AllowedIPs = 10.0.0.2/32

5. System Configuration

Enabled IP forwarding:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Allowed WireGuard port:

```
sudo ufw allow 51820/udp
```


Added Azure inbound rule:

- Port: 51820
 - Protocol: UDP
 - Source: Any
-

6. Starting WireGuard

```
sudo wg-quick up wg0
```

7. Issues Faced & Fixes

Issue 1: Connection Timeout

- **Cause:** Azure firewall / endpoint issues
 - **Fix:** Verified correct public IP and ensured UDP 51820 was allowed
-

Issue 2: No Handshake

- **Cause:** Incorrect or unloaded peer configuration
- **Fix:** Restarted WireGuard:

```
sudo wg-quick down wg0
```

```
sudo wg-quick up wg0
```

👉 Learned that WireGuard does NOT auto-reload configs.

Issue 3: Key Mismatch Confusion

- Mixed up public/private keys between server and client.
- **Fix:** Correct mapping:

Location	Key
Server Interface	Server Private Key
Server Peer	Client Public Key
Client Interface	Client Private Key
Client Peer	Server Public Key

Issue 4: VPN Connected but No Internet / Slow

- **Cause:** Full tunnel routing (0.0.0.0/0)

Traffic was going:

Laptop → VM → Internet

-
- **Fix:** Switched to split tunnel:

AllowedIPs = 10.0.0.0/24

Issue 5: SSH Access Issues

- SSH failed when full tunnel enabled
- **Reason:** Traffic routed through VPN incorrectly
- **Fix:** Use split tunnel or connect via VPN IP (10.0.0.1)

8. Client Configuration (Windows)

Used WireGuard GUI.

[Interface]

PrivateKey = <client1_private.key>

Address = 10.0.0.2/24

[Peer]

PublicKey = <server_public.key>

Endpoint = <VM_PUBLIC_IP>:51820

AllowedIPs = 10.0.0.0/24

PersistentKeepalive = 25

9. Verification

VPN connectivity test:

ping 10.0.0.1

Result:

- Successful replies
- No packet loss

WireGuard status:

sudo wg

Confirmed:

- Peer detected
 - Handshake successful
 - Data transfer active
-

10. Final Outcome

Successfully achieved:

- WireGuard VPN server setup
 - Two client configurations created
 - Secure communication established
 - VPN tunnel verified using ICMP and handshake logs
-

11. Key Learnings

- WireGuard does not auto-reload configs → requires restart
 - Correct key pairing is critical
 - NAT is required for internet routing
 - Full tunnel vs split tunnel affects performance and routing
 - Cloud firewall rules must allow UDP traffic explicitly
-

12. Conclusion

The VPN setup was successfully implemented and tested. The system ensures secure communication between client and server, fulfilling all requirements of the task.
